

Red Agent Plan

Nexus Workspace — Autonomous Security Assessment

Date: September 2026 | Target: Nexus Workspace SaaS Environment | Method: Formula-Based Discovery

1. The Red Agent's Job

Goal: Starting as a regular user with zero knowledge of the system, reach the `/internal/api-keys` endpoint that only a "system" role can access.

Constraint: The agent has no insider knowledge. It does not read config files, source code, or documentation. It learns entirely through interaction.

2. How It Thinks

Phase 1 — Map the Territory

The agent starts with no access and registers an account. It now has a token and a basic user identity. From here, it begins a systematic survey.

START: - Register → get token → what does this token allow? For every endpoint it can call: - What did I send? (request) - What came back? (response) - Was this what SHOULD have come back? Unexpected responses are the agent's bread and butter: - 200 when it expected 403 → something is too permissive - More data returned than expected → information disclosure - Less data returned → something changed, worth investigating - Slower response → something is happening behind the scenes - Error message contains a secret URL or token → straight to findings

The agent doesn't know what "unexpected" means by label. It measures it mathematically:

Expected response = what a properly secured endpoint returns Actual response = what the endpoint actually returned $\Delta = \text{actual} - \text{expected}$ If $|\Delta| > \text{threshold}$: → Flag as anomaly → Investigate deeper

Phase 2 — Build the Permission Graph

As the agent explores, it maps the system as a **graph of capabilities**:

NODE = a permission level (guest, user, admin, system) EDGE = an action that moves the agent between nodes The agent's job: find a PATH from user → system But it doesn't know where the edges are yet. It discovers them by trying things.

The graph is built purely from observations:

Observation 1: Call GET `/users/me` → I see my own user ID and role Call GET `/users/{other_user_id}` → I got their data Δ = I was NOT supposed to see other users' data Edge found: Guest/User can enumerate all users Observation 2: Call POST `/api-keys` → Got a new API key with my scopes Call PUT `/api-keys/{key_id}` → Changed scopes to ["admin"] Δ = I was NOT supposed to be able to escalate my own scopes Edge found: User can become admin via API key modification Observation 3:

Call POST /integrations/connect → Integration connected Webhook fires internally
→ Check webhook logs Δ = Webhook payload contains tokens that were never visible
in the API Edge found: Integration interaction exposes hidden data

Each observation alone is a data point. The agent chains them.

Phase 3 — Chain Low-Severity into High-Severity

This is the core of how the agent thinks. It doesn't look for one big vulnerability — it looks for **chains of small permissions that combine into a big reach**.

The agent chains observations together, using each one as a stepping stone to the next:

Step 1: Start as regular user → Can view any user profile (IDOR) → Knows admin's user ID
Step 2: Create an API key → Key has limited scopes by default → Can modify own API key scopes (Privilege Escalation) → Upgrades key to ["admin"]
Step 3: With admin scopes, call /admin/settings → Admin settings are exposed to admin role → Reveals Stripe webhook secret and OAuth credentials
Step 4: With Stripe webhook secret, forge billing events → Webhook endpoint accepts events without verifying sender → Escalate own account to admin via forged event
Step 5: With admin role, call /admin/audit → Audit log shows internal API key table structure → Shows which users have system-level access
Step 6: With knowledge from audit log + admin access, call /internal/api-keys → TARGET REACHED

The agent doesn't know these steps in advance. It discovers them one by one, measuring deviations at each step.

Phase 4 — The Deviation Formula

For every action the agent takes, it measures:

METRIC	FORMULA
Status Deviation	actual_status ≠ expected_status
Data Leak	actual_response > expected_size
Time Anomaly	response_time > baseline × 2
Scope Leak	response reveals hidden tokens
Access Bypass	403/401 → 200 on retry
Data Enumeration	sequential IDs return valid data
Role Confusion	user role acts as admin role

"This is outside the normal distribution. Worth investigating."

Phase 5 — Reaching the Target

The agent knows it's reached the target when:

Target condition: Response from endpoint contains ALL user API keys AND role in response = "system"
The agent doesn't stop at intermediate wins. It keeps going

until it hits the actual crown jewels.

3. The Agent's Operational Loop

The agent repeats this loop until the target is reached:

LOOP until target reached: 1. PROBE Send a request to every known endpoint Measure response against baseline 2. CATALOG Build a list of: what's accessible, what's not, what's partially accessible 3. COMBINE Try using two capabilities together: - API key + user profile = escalate scope - Webhook + token = replay attack - File upload + file ID = enumerate other users' files 4. ESCALATE If a combination opened a new permission level: - Re-map all accessible endpoints from new level - Repeat from step 1 5. VALIDATE If target endpoint returns data: - Confirm it's the real target (not a decoy) - Report the full attack chain

4. How It Discovers Without Being Told

The agent never knows:

- Which endpoints are vulnerable
- Which integrations interact dangerously
- Where tokens are stored
- What the permission model looks like

It only knows:

- I can call this endpoint - This is what came back - This is different from what a properly secured system would return - Let me try combining this with something else

The vulnerabilities emerge because:

"The system is complex enough that nobody tested every combination of every integration. The agent tests every combination systematically."

5. Summary

Concept	How the Agent Uses It
API Exploration	Calls every endpoint, measures every response
Permission Graph	Maps what each role can access by trying
Deviation Detection	Compares actual vs expected, flags the delta
Chain Building	Connects small edges into a path to system
No Prior Knowledge	Learns entirely from system responses

The vulnerabilities are not in the code — they are in the *interaction* of capabilities that nobody tested together. The agent finds them the same way a real attacker would: by trying things, observing what

breaks, and following the cracks.

6. Key Principles

- **No Hard-Coding** — The agent has no list of vulnerabilities. It discovers them through exploration.
- **Emergent Discovery** — Vulnerabilities emerge from the interaction of multiple components, not from individual features.
- **Chain Building** — Low-severity findings combine into high-severity compromise. No single step is the attack.
- **Measurement-Based** — Every anomaly is quantified, not assumed. The agent proves each deviation statistically.
- **Goal-Oriented** — The agent works toward the target endpoint, but explores freely. The path to the target is discovered, not prescribed.